SOLUTION ARCHITECTURE · SEARCH INTERNALS · JVM TUNING

Magento + Elasticsearch

How Magento Gets Faster Search Results

From EAV catalog data to inverted-index search: architecture, indexing, relevance, aggregations, JVM memory, and production troubleshooting.

Presented by

Senior Adobe Commerce Solution Architect

13+ Years E-Commerce Architecture & Delivery · Magento 2 / Adobe Commerce, Elasticsearch & OpenSearch

ADOBE COMMERCE | MAGENTO 2 | ELASTICSEARCH / OPENSEARCH

SECTION 1

Why MySQL Alone Isn't Enough for Search

Catalog Data Lives in EAV Tables

Magento's flexible product model spreads a single product's attributes across several entity-attribute-value tables by data type.

Full-Text Relevance Isn't MySQL's Job

Relational tables are built for structured lookups, not for ranking free-text matches by relevance.

Layered Navigation Needs Aggregation

Faceted counts (Brand: Nike (120), Adidas (95)…) require aggregating across the full result set — expensive to compute repeatedly in MySQL.

Magento Can Still Perform Well

With proper indexing, caching, and database tuning Magento's own data layer performs well — full-text and faceted search are simply a different, specialized use case.

One Product, Several EAV Tables

```
catalog_product_entity
catalog_product_entity_varchar
catalog_product_entity_int
catalog_product_entity_decimal
catalog_product_entity_text
```

A full-text search must reach across many of these tables and joins to evaluate a single query.

SECTION 1 · FRAMING

The Magento Search Problem

A customer searches for "running shoes." What does Magento actually need to do, well, in real time?

Match Relevance

Rank "running shoes" above loosely related items, even with typos or partial terms.

Apply Filters

Narrow by brand, color, size, and price simultaneously.

Compute Aggregations

Show live facet counts for every available filter combination.

Respond Instantly

Return results fast enough to feel native to the storefront.

All four of these at once, on every keystroke-driven request, across a catalog that keeps growing — this is exactly the workload a dedicated search engine is designed for.

SECTION 1 · THE SOLUTION

Enter Elasticsearch / OpenSearch

Purpose-Built for Search

Built around an inverted index designed specifically for full-text relevance.

Distributed by Design

Scales horizontally across shards and nodes as data and traffic grow.

Relevance Scoring Built In

Ranks matches by relevance automatically, not as an afterthought.

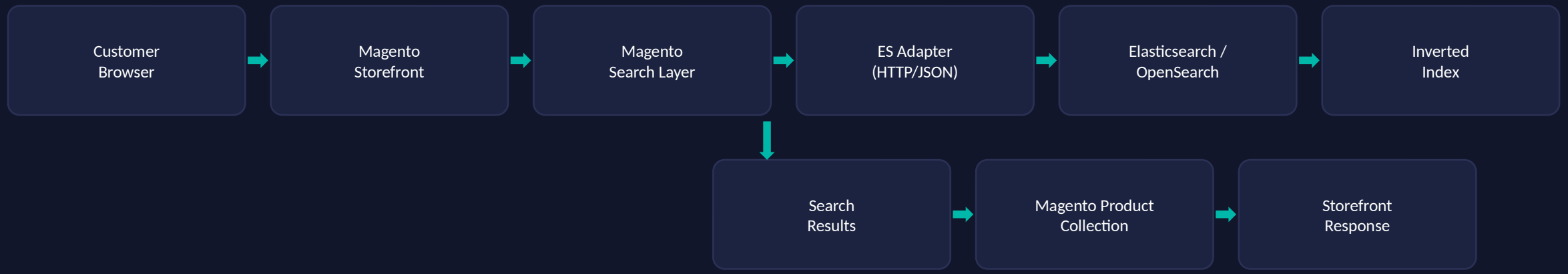
Native Aggregations

Computes facet counts and analytics as part of the same query.

Important distinction: Elasticsearch is not the primary Magento database. It is a search engine / search index Magento uses alongside MySQL — MySQL remains the source of truth.

ADOBE COMMERCE | MAGENTO 2 | ELASTICSEARCH / OPENSEARCH

Magento + Elasticsearch Architecture



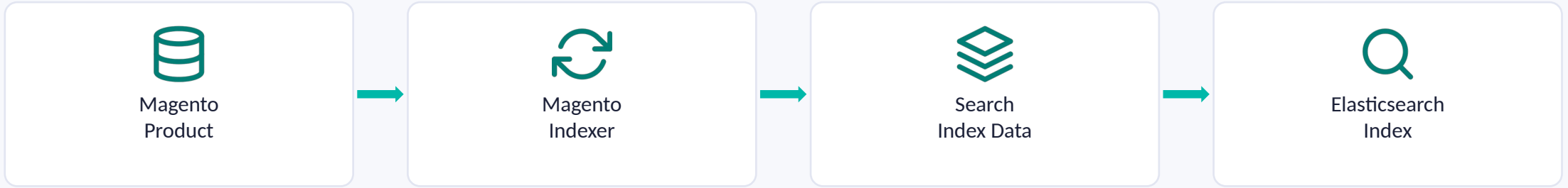
Keeping the Index Current



Elasticsearch is NOT the primary Magento database — it is a search index derived from MySQL via Magento's indexers.

SECTION 6

How Magento Indexing Works



What Triggers Reindexing

- Product create / update
- Category changes
- Attribute changes
- Price changes
- Stock changes

Indexer Modes

Update on Save: reindexes immediately, adding save-time overhead.

Update by Schedule: cron-driven, batched — the typical production choice.

```
$ bin/magento indexer:status
$ bin/magento indexer:show-mode
$ bin/magento indexer:reindex
```

SECTION 3

What Happens During a Search

Example query: "nike running shoes"

1. Customer enters search term
2. Magento receives the request
3. Search module builds a search request
4. Search adapter builds the Elasticsearch query
5. Magento sends a JSON request to Elasticsearch
6. Elasticsearch analyzes the text
7. Elasticsearch searches the inverted index
8. Elasticsearch calculates matching documents
9. Elasticsearch applies filters
10. Elasticsearch performs aggregations
11. Elasticsearch returns matching document IDs
12. Magento loads full product data for those IDs
13. Storefront renders the results

```
{ "query": { "bool": { "must": [
  { "multi_match": { "query": "nike running shoes",
      "fields": ["name","sku","description"] } }
] } } }  // simplified — not necessarily exact per version
```

SECTION 4

Inverted Index Explained

Documents

Product 101: Nike Running Shoes

Product 102: Adidas Running Shoes

Product 103: Nike Casual Shoes

Elasticsearch tokenizes each document's text into terms and records which documents contain each term.

Inverted Index (Term → Documents)

`nike`	→ 101, 103
`running`	→ 101, 102
`shoes`	→ 101, 102, 103
`adidas`	→ 102
`casual`	→ 103

A query for "nike" jumps straight to documents 101 and 103 — no scanning required. This term-to-document lookup is why search engines resolve full-text queries so efficiently versus repeatedly scanning relational/EAV tables.

SECTION 5

Magento EAV vs. Elasticsearch Document

Magento EAV — Source Catalog Model

```
catalog_product_entity
catalog_product_entity_varchar
catalog_product_entity_int
catalog_product_entity_decimal
catalog_product_entity_text
...
```

Optimized for flexible, typed commerce data — not for full-text relevance across many products at once.

Elasticsearch — Search Document

```
{
  id, sku, name, price,
  category, attributes,
  searchable content,
  filterable fields, ...
}
```

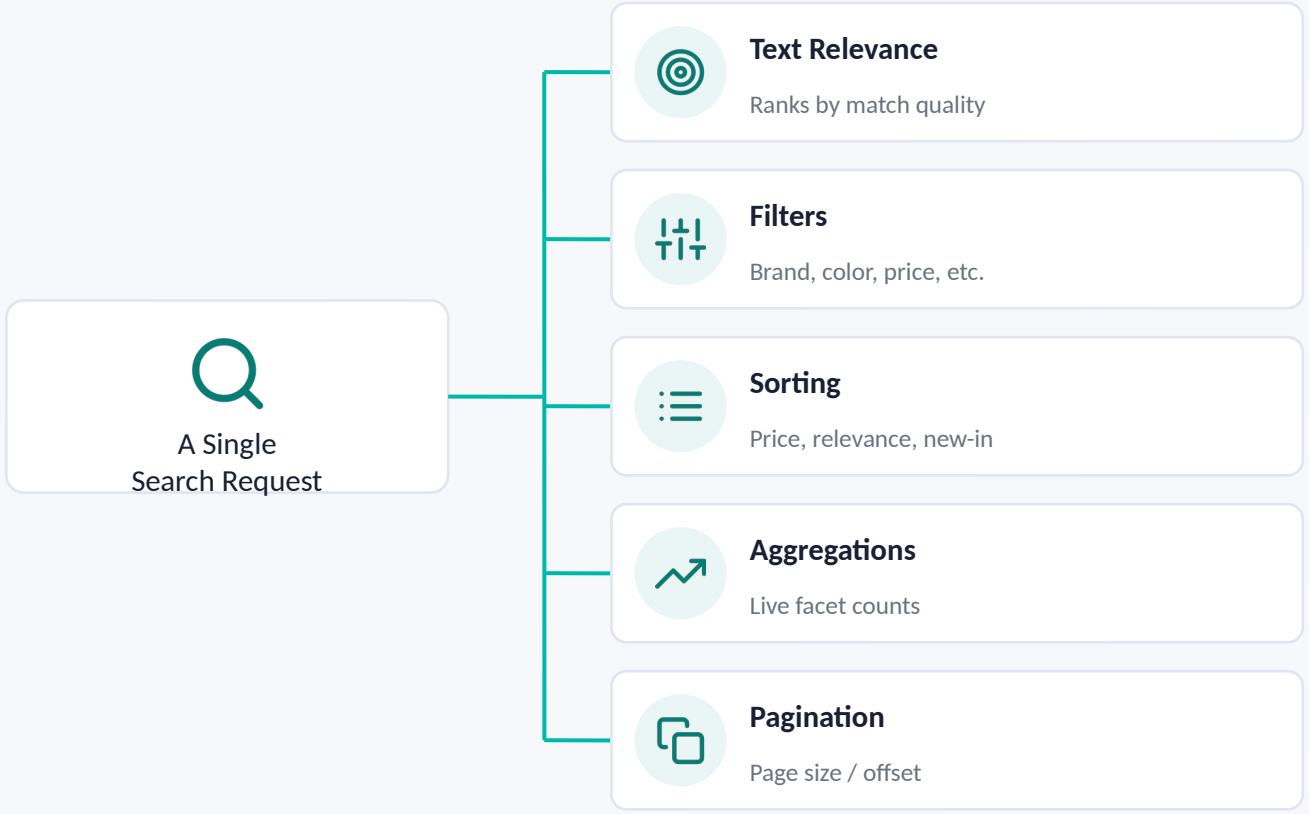
Optimized for search and retrieval — one flattened, denormalized document per product.

Not "EAV is slow" — each model is optimized for a different job. Magento performs well with proper indexing/caching; search and faceted discovery are where a dedicated engine adds the most value.

SECTION 17

Search + Filter + Aggregation

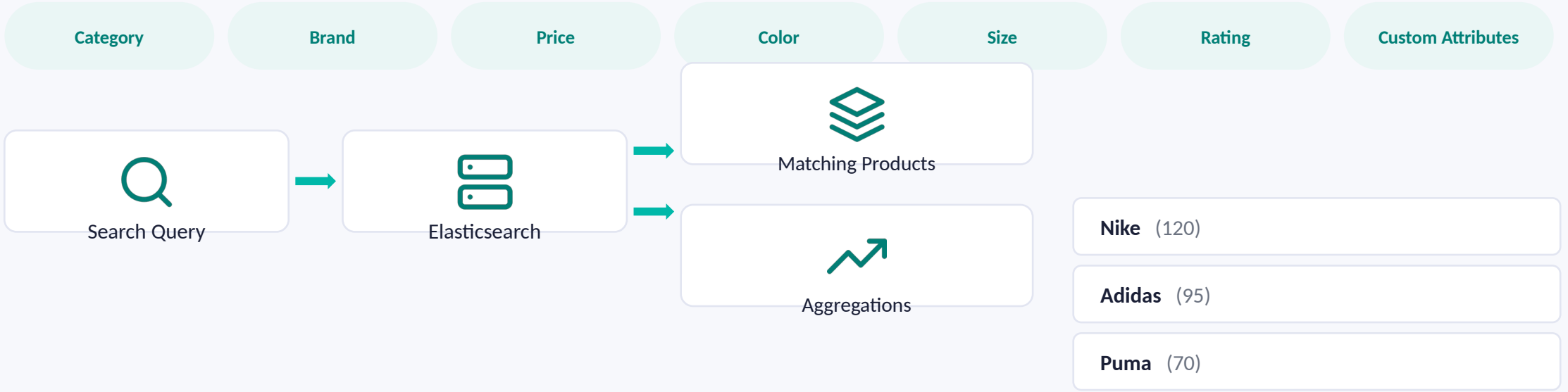
Full-text search *("running shoes")* finds relevant matches. **Filters** (brand = Nike, color = Black, price < 5000) narrow the set precisely — without affecting relevance scoring.



SECTION 18

Magento Layered Navigation

Elasticsearch aggregations power every filter panel on the category and search results page.

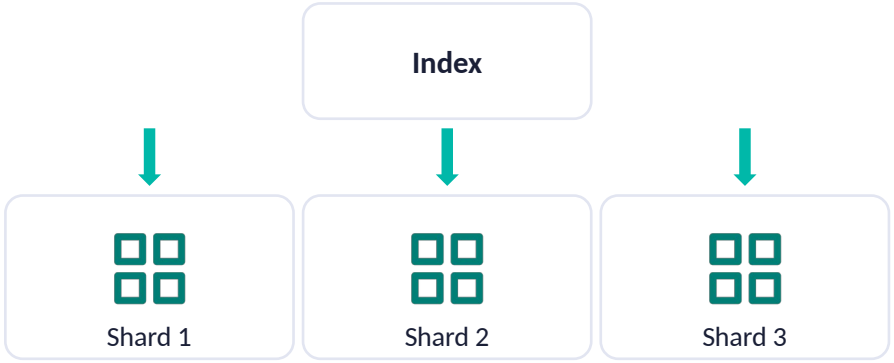


These facet counts are computed by Elasticsearch aggregations in the same request that returns matching products — not a separate query per filter.

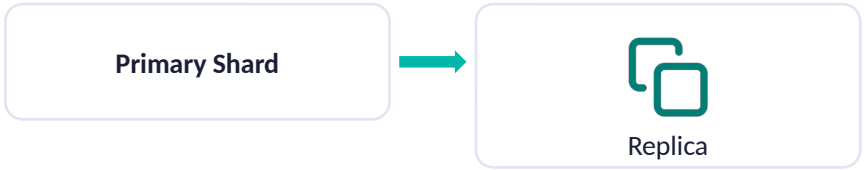
SECTION 16

Shards & Replicas

An Index Is Split Into Shards



Shards let a single index scale across multiple nodes and parallelize work.



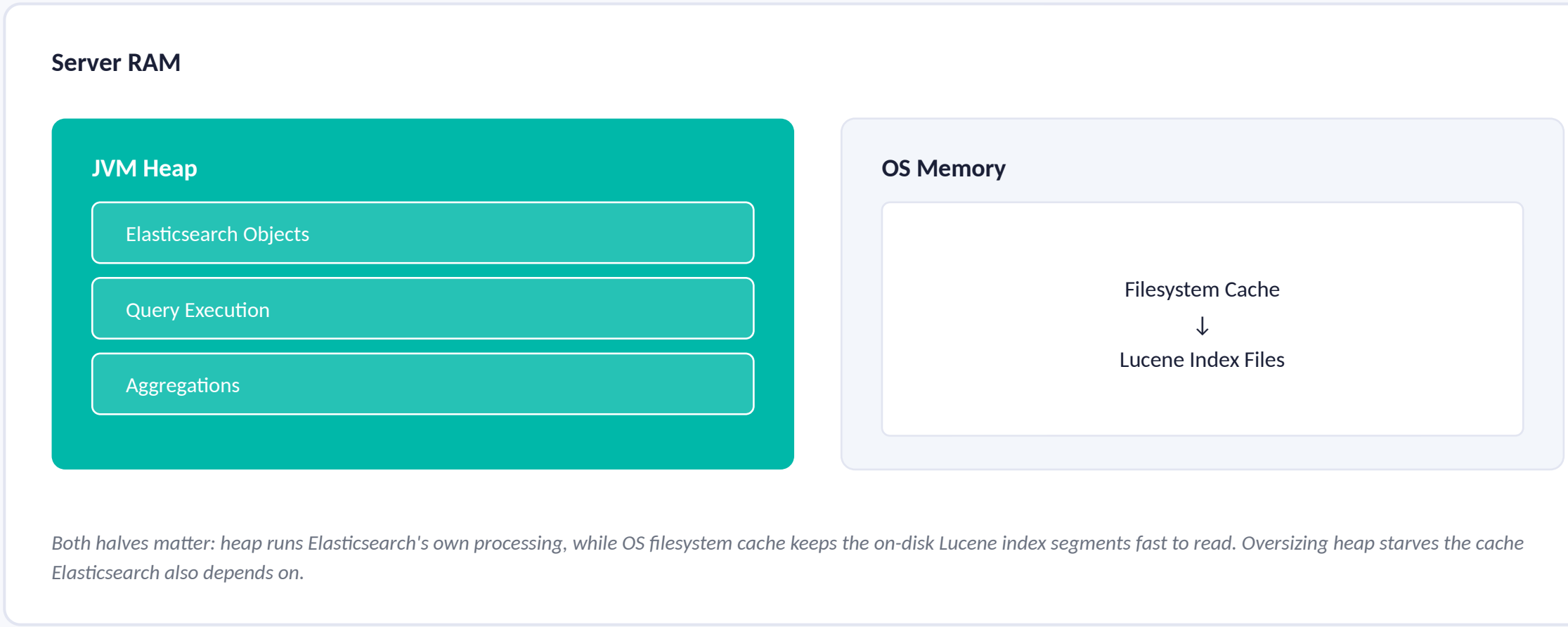
Why Each Exists

- Shards distribute data and query load across nodes
- Replicas provide availability if a node is lost
- Replicas can also serve reads, improving throughput for appropriate workloads
- Too many shards adds coordination overhead and hurts performance
- Shard count is typically planned up front — it is not easily changed later

```
$ curl "http://localhost:9200/_cat/shards?v"
$ curl "http://localhost:9200/_cat/indices?v"
```

SECTION 15

JVM & Memory Architecture



Both halves matter: heap runs Elasticsearch's own processing, while OS filesystem cache keeps the on-disk Lucene index segments fast to read. Oversizing heap starves the cache Elasticsearch also depends on.

JVM Options — Xms / Xmx

```
-Xms1g
-Xmx1g
```

Xms = initial / minimum heap size

Xmx = maximum heap size

Commonly set equal so the JVM doesn't resize the heap at runtime.

⚠ Do NOT blindly recommend 50% of RAM or any fixed production value. Sizing depends on ES/OpenSearch version, available RAM, workload, shard count, index size, JVM version, OS, filesystem-cache needs, and concurrent query/aggregation load.

Before / After Experiment — Measure, Don't Assume

Scenario A -Xms512m -Xmx512m	Scenario B -Xms1g -Xmx1g

- Heap usage
- GC activity
- Query latency
- Indexing throughput
- Node stability

More heap does not automatically mean better performance — and oversized heap starves the filesystem cache Elasticsearch relies on.

SECTION 19–20

Performance Troubleshooting

Magento Can't Connect to Elasticsearch

- Wrong hostname
- Wrong port
- Authentication failure
- TLS/SSL misconfiguration
- Elasticsearch unavailable
- Incompatible version

- Index missing / stale
- Indexing failure
- Insufficient JVM heap
- Disk watermark reached
- Shard problems

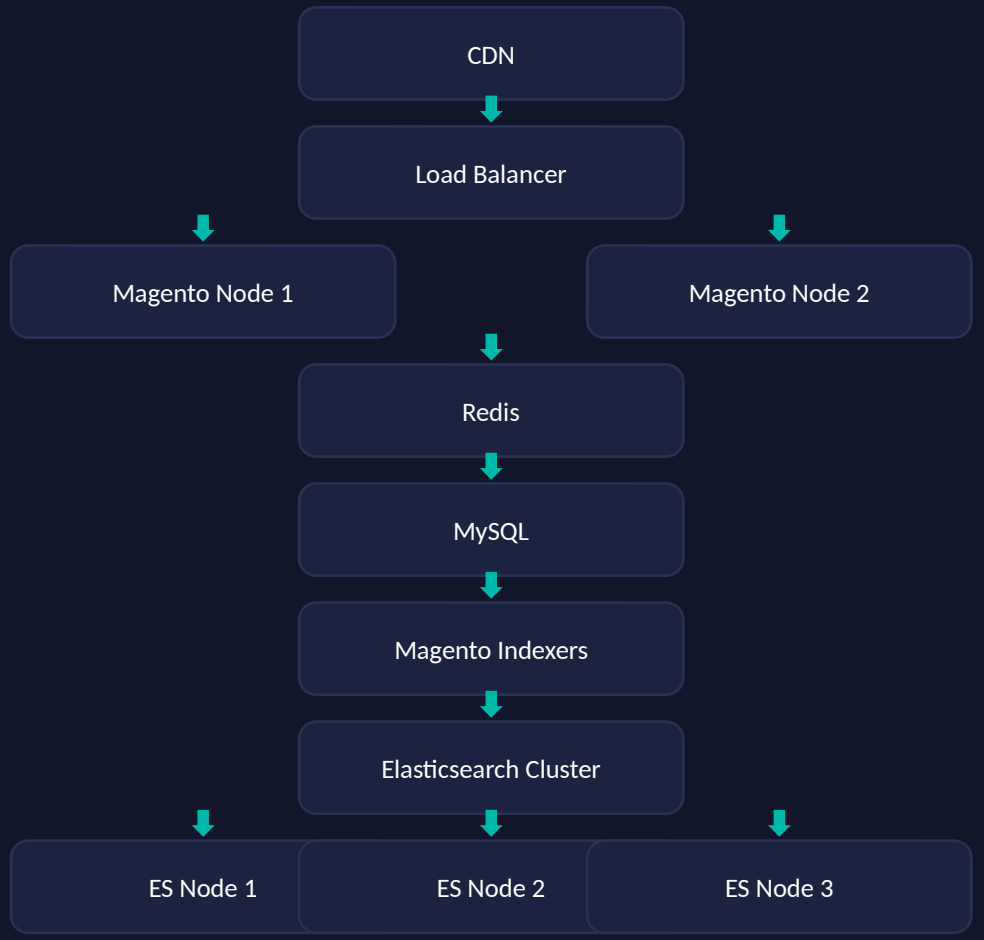
Diagnostic Commands

```
$ curl localhost:9200
$ curl localhost:9200/_cluster/health?pretty
$ curl localhost:9200/_cat/indices?v
$ curl localhost:9200/_cat/shards?v
$ curl localhost:9200/_nodes/stats?pretty
```

```
$ bin/magento indexer:status
$ bin/magento indexer:show-mode
$ bin/magento cache:status
```

Isolate the layer first: search engine, Magento indexing/cache, or database — before changing any configuration.

400K+ Product Architecture



Scenario

400,000+ products
100,000+ customers
Many attributes & categories
High traffic

Architecture Considerations

- Shard planning
- Replica strategy
- JVM sizing
- Index refresh interval
- Indexing strategy
- Bulk operations
- Incremental indexing
- Query optimization
- Aggregation cost
- Infrastructure sizing

SECTION 23

Live Demo Script (10–15 Minutes)

1 Check Elasticsearch

```
curl http://localhost:9200
```

2 Cluster health

```
curl .../_cluster/health?pretty
```

3 Magento indexer status

```
bin/magento indexer:status
```

4 Inspect indexes

```
curl .../_cat/indices?v
```

5 Search

```
Query: "running shoes"
```

6 Filter

```
brand = Nike, color = Black
```

7 Aggregation

```
Nike / Adidas / Puma counts
```

8 Inspect JVM

```
curl .../_nodes/jvm?pretty
```

9 Change JVM config

```
Edit Docker heap, restart
```

10 Explain results

```
What changed, what didn't
```

SUMMARY

Summary & Production Checklist



Source of Truth → Transform / Prepare → Fast Search, Filtering, Aggregations, Relevance → Storefront

Magento uses Elasticsearch not because MySQL can't store products, but because a dedicated search engine is optimized for how customers search, filter, and discover products.

Production Checklist

- ✓ Verify version compatibility (Magento ↔ ES/OpenSearch)
- ✓ Heap sized to workload — never a blind 50% rule
- ✓ Shard/replica plan set for expected catalog size
- ✓ Indexer mode and cron health confirmed
- ✓ Never expose Elasticsearch publicly without securing it
- ✓ No hardcoded credentials — environment variables only
- ✓ Benchmarks measured, never assumed

Questions?

ADOBE COMMERCE | MAGENTO 2 | ELASTICSEARCH / OPENSEARCH